Pseudocode to Java Compiler Phase 3 Final Report

Compiler Design Laboratory

Student: Roshani Verma

Programme: B.Tech CSE (Blockchain), VIT

Submission phase: Phase 3 Final Implementation and Testing

Date: 23 October 2026

Abstract

This report documents a hand-written compiler that translates a typed pseudocode language into Java, compiles the generated source, and executes it. The Phase 3 work adds a Swing visualizer that refreshes tokens, the abstract syntax tree (AST), scope information, and generated Java while the source is edited; a semantic diagnostic utility that reports independent errors across top-level constructs; and a reproducible performance runner. The original Phase 2 regression suite and new Phase 3 checks pass together: 148 passed and 0 failed. The report follows the chapter structure in the Compiler Design Laboratory project instruction manual.

Contents

1. Chapter 1: Introduction

2. Chapter 2: Problem Statement

3. Chapter 3: Objectives

4. Chapter 4: Literature Survey or Background Study

5. Chapter 5: System Design

6. Chapter 6: Methodology

7. Chapter 7: Compiler Design Concepts Used

8. Chapter 8: Implementation

9. Chapter 9: Testing and Results

10. Chapter 10: Conclusion

11. Chapter 11: Future Enhancements

12. References

13. Appendix

Chapter 1: Introduction

Compiler design connects a source language to executable behavior through a sequence of well-defined transformations. Lexical analysis groups characters into tokens; parsing checks grammatical structure and builds a syntax tree; semantic analysis checks declarations, types, and scopes; code generation maps the validated program to a target language. These stages are often studied separately, which can make it difficult to see how a source statement becomes running code.

This project makes that transformation observable. A user writes a small pseudocode program, inspects the token stream and AST, sees the semantic and symbol-table information, and can view the generated Java and its execution result. The compiler is written in Java without parser generators or third-party runtime libraries. Phase 3 completes the planned user interface, semantic error recovery, performance measurements, test evidence, and documentation around the Phase 2 compiler.

Chapter 2: Problem Statement

Students commonly develop algorithms in pseudocode before implementing them in a programming language. Manual translation can introduce mistakes, while a conventional compiler hides the internal steps that detect those mistakes and transform the source. The problem addressed here is to provide a working pseudocode-to-Java compiler that both translates programs and exposes its stages for inspection.

The system must recognize a defined language, validate syntax and semantics, manage nested scopes, generate Java, and execute the generated program. It must also provide a live view of compiler stages, useful source-positioned errors, a way to report multiple independent semantic errors, and repeatable evidence that the implementation behaves as intended.

Chapter 3: Objectives

- Implement a hand-written lexer and recursive-descent parser for the project language.
- Represent parsed programs as an AST and use it as the compiler's intermediate representation.
- Check types, variable declarations, function calls, returns, and nested scope rules.
- Generate readable Java, compile it with the JDK, and execute it.
- Visualize tokens, AST, scope and symbol information, and generated Java during editing.
- Recover from independent semantic errors in separate top-level constructs.
- Validate the compiler with unit and end-to-end programs, including valid, invalid, boundary, and regression cases.
- Measure front-end/code-generation time and full Java compile-and-run time.
- Document build, run, test, diagnostic, and benchmark procedures.

Chapter 4: Literature Survey or Background Study

The design follows the standard compiler pipeline described in compiler-construction literature: lexical analysis, parsing, semantic analysis, intermediate representation, and target-code generation. A recursive-descent parser is appropriate for this compact language because the grammar is organized into explicit precedence levels; each parser method corresponds to one grammar rule. The AST is a useful intermediate representation because it separates source syntax from semantic checking and target generation.

The project proposal supplied with the assignment identifies compiler visualization as a way to make intermediate representations and symbol-table changes easier to understand. The Phase 2 implementation already printed tokens, ASTs, scopes, generated Java, and real output in a command-line run. Phase 3 extends that demonstration into an interactive Swing editor and adds error recovery and performance measurement.

The implementation uses Java SE APIs: Swing provides the desktop widgets and event model; the JDK compiler/runtime tools compile and execute the generated Java. The project itself does not use a parser generator or external libraries. Its design keeps the lexer, parser, AST, analyzer, symbol table, generator, executor, display, and tests in separate modules.

Chapter 5: System Design

5.1 Architecture

Data path: Source editor → Lexer → Tokens → Parser → AST → Semantic analyzer and symbol table → Java generator → JDK compilation → Program execution.

The visualizer observes the intermediate data produced by the compiler stages. Diagnostics and performance tools reuse the compiler's public stage APIs. The command-line entry point remains available independently of the visualizer.

5.2 Module description

Module	Responsibility	Main source files
Lexer	Recognizes keywords, identifiers, literals, operators, punctuation, and source positions	Lexer.java, Token.java, TokenType.java
Parser and AST	Parses statements and precedence-based expressions into typed node classes	Parser.java, ASTNode.java, node classes
Semantic analysis	Validates declarations, scopes, types, calls, returns, and conditions	SemanticAnalyzer.java, CompileError.java
Symbol table	Stores variable bindings by nested scope and function signatures	SymbolTable.java
Code generation	Emits readable Java while preserving types and name resolution	CodeGenerator.java
Execution	Invokes the JDK compiler and runs the generated class	Executor.java
Visualization	Updates the source editor's stage panels and executes valid output	CompilerVisualizer.java, AstPrinter.java, TablePrinter.java
Phase 3 support	Collects independent semantic diagnostics and measures performance	DiagnosticsCli.java, PerformanceRunner.java
Validation	Runs unit checks and 100 end-to-end programs	TestRunner.java, tests/

5.3 Data flow and workflow

1. Read source from a file, the visualizer editor, or the built-in example.
2. The lexer emits positioned tokens; invalid characters or malformed literals produce lexical diagnostics.
3. The parser consumes tokens and builds a program AST using explicit operator-precedence methods.
4. The semantic analyzer checks variable and function declarations, expression types, nested scopes, and return paths while recording symbol-table events.
5. The generator emits a Java class. The executor invokes the JDK compiler and runs the class.
6. The visualizer refreshes each successful stage as the text changes; the test and diagnostics tools report results in the terminal.

5.4 User interface

The Swing window places the source editor beside panels for tokens, AST, symbol/scope details, and generated Java. A short debounce delay avoids recompiling after every keystroke. Open file and Save source controls handle local files. Compile and run sends execution to a background worker to keep the interface responsive. When semantic analysis finds errors, the diagnostics area lists recoverable independent semantic errors.

Chapter 6: Methodology

Development continued from the working Phase 2 compiler instead of replacing its pipeline. The implementation sequence was: inspect existing APIs and tests; add the visualizer as a separate client of the compiler stages; add semantic recovery by retrying analysis after excluding the failing top-level construct; add a benchmark with warm-up and repeated measurements; extend unit checks; update user documentation and the final report; then rebuild, run regression tests, and package the source and evidence.

The recovery utility starts with a successfully tokenized and parsed program. It analyzes the set of top-level constructs; when a semantic error occurs, it records the positioned diagnostic and removes the top-level construct containing that error. It then creates a fresh analyzer and symbol table for the remaining constructs. Reinitializing semantic state prevents a failed analysis from leaving partially entered scopes or declarations behind.

The benchmark separates compiler front-end and code-generation time from full end-to-end time. It performs three warm-up rounds, then records ten runs by default. Results include mean, median, minimum, and maximum. End-to-end timing includes generated Java compilation and execution, so it also reflects process startup and the local JDK and machine load.

Chapter 7: Compiler Design Concepts Used

• **Lexical analysis:** a hand-written scanner recognizes 39 token kinds, skips comments and whitespace, decodes string escapes, and tracks line and column.

• **Syntax analysis:** recursive-descent methods implement the grammar and operator precedence; syntax errors identify expected input and source location.

• **AST and intermediate representation:** one node type represents each supported statement or expression; external passes traverse the tree.

• **Semantic analysis:** type checking, declaration-before-use, function signatures, return validation, and boolean-condition checks run before generation.

• **Symbol-table management:** a scope stack supports lookup, shadowing, loop/block/function scope boundaries, and a separate function signature table.

• **Target-code generation:** the AST is translated to Java. Pseudocode types map to Java types; global variables become static fields; shadowed locals are renamed where Java's local naming rules require it.

• **Execution:** generated Java is compiled and run by the JDK, providing concrete output rather than a translation-only demonstration.

• **Error detection and recovery:** all existing compile errors carry a stage, line, and column. Phase 3 recovery collects independent semantic errors across top-level constructs.

• **Optimization:** there is no separate optimization pass in this project. The generator emits readable Java with precedence-aware parentheses; this is code generation, not a claim of performance optimization.

Chapter 8: Implementation

8.1 Tools and technologies

Item	Choice
Language	Java
Minimum documented runtime	JDK 17+
Tested runtime for this report	JDK 25.0.4.1
User interface	Java Swing
Parser approach	Hand-written recursive descent
Build	javac command line
Test framework	Dependency-free Java TestRunner
External runtime libraries	None

8.2 Language and features

The language supports optional type annotations and inference for LET declarations; int, float, boolean, and string types; assignment; arithmetic, relational, equality, and logical expressions; if/else/else-if, while, and for statements; typed function parameters and returns; PRINT and INPUT; nested blocks; and line comments. Phase 2 behavior remains available through the command-line compiler.

8.3 Phase 3 modules

CompilerVisualizer.java creates the editable Swing interface and updates token, AST, symbol, and generated-source panels using the existing compiler classes. **DiagnosticsCli.java** exposes the recovery algorithm for a source file; the same diagnostic list is displayed in the visualizer for semantic failures. **PerformanceRunner.java** performs a reproducible sample run and prints timing summaries. **TestRunner.java** now includes checks for diagnostic count and source order.

Chapter 9: Testing and Results

9.1 Test approach

The suite includes unit checks for the lexer, parser, symbol table, semantic analyzer, generator, and Phase 3 recovery. End-to-end fixtures exercise valid programs, error cases, scope behavior, edge cases, and Phase 1 regressions. The final validation rebuilt every source file and ran the complete test runner after the visualizer's diagnostic integration.

Test group	Passed	Total	Coverage
Unit tests	48	48	Compiler modules and Phase 3 recovery count/order
Valid programs	29	29	Types, functions, recursion, loops, input/output, logic, strings
Scope	11	11	Shadowing and block/loop/function scope behavior
Syntax errors	17	17	Lexical and syntax failures with expected locations/messages
Semantic errors	27	27	Type, declaration, function, return, and condition errors
Edge cases	13	13	Empty blocks, precedence, literal, and runtime cases
Phase 1 regression	3	3	Earlier programs remain compatible
Total	**148**	**148**	**0 failures**

9.2 Output evidence

The independent error example was run with the Diagnostics CLI. The captured output was:

```
3 semantic error(s):

Semantic error at line 1, column 19: Variable 'missing' used before declaration

Semantic error at line 2, column 24: Type mismatch in declaration of 'second': expected 'boolean' but found 'int'

Semantic error at line 3, column 19: Type mismatch in declaration of 'third': expected 'int' but found 'string'

Recovery is top-level: errors inside one top-level construct are reported once.
```

This output demonstrates that independent top-level errors are reported in one run and include source locations. A valid factorial program is included in the visualizer's startup source and the existing end-to-end examples; the full test suite confirms generated Java behavior across the valid fixtures.

9.3 Performance results

The bundled benchmark used an integer value, a function call, and output; it ran three warm-up rounds followed by ten measured iterations on JDK 25.0.4.1.

Measurement	Mean	Median	Minimum	Maximum
Lexer, parser, semantic analysis, code generation	0.973 ms	1.026 ms	0.406 ms	1.618 ms
Full pipeline, generated Java compilation, and execution	1,030.475 ms	1,042.038 ms	600.390 ms	2,079.566 ms

These are measurements from one host and one sample program. End-to-end timings include compiler and process startup. The values are intended as a repeatable baseline, not as a hardware-independent guarantee. The benchmark does not measure memory use.

9.4 Limitations of the results

The GUI was compiled as part of the project; the report's executable evidence focuses on the full test runner, diagnostic CLI, and benchmark output. Lexical and parser diagnostics remain fail-fast. Semantic recovery is top-level: it reports a nested failure once and can expose dependent errors if a failed declaration is removed. The performance sample is small and measures one representative program.

Chapter 10: Conclusion

The project integrates a hand-written compiler pipeline with Java code generation and execution, and Phase 3 adds an interactive stage viewer, independent semantic error collection, repeatable performance measurement, and final instructions. The final build passed all 148 checks, including 100 end-to-end fixtures and 48 unit checks. The delivered project archive contains the source, tests, examples, user guide, and this report in both PDF and editable Markdown form.

Chapter 11: Future Enhancements

• Add parser-level synchronization to recover from multiple syntax errors in one pass.

• Improve semantic recovery so dependent errors are filtered or linked to their root cause.

• Add token-by-token stepping and execution-state visualization.

• Add a representative benchmark corpus, memory measurements, and comparative baselines.

• Capture screenshots of the visualizer on supported desktop platforms and include them in future review evidence.

• Add a dedicated optimization pass only if the language and evaluation scope require it, with before/after correctness and performance measurements.

References

1. A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed., Pearson, 2006.

2. Compiler Design Laboratory, *Project Instruction Manual*, supplied PDF, sections 12 and 24 (Phase 3 activities/deliverables and documentation format).

3. Roshani Verma, *Pseudocode-to-Java Compiler: Phase 1 Project Proposal*, supplied assignment document AST05.

4. Roshani Verma, *Pseudocode-to-Java Compiler: Phase 2 Project Proposal — Core Implementation*, supplied assignment document AST07.

5. Roshani Verma, *Pseudocode-to-Java Compiler: Phase 2 Review*, supplied assignment document AST08.

6. Oracle, *Java SE 25 API Specification: javax.swing package*, https://docs.oracle.com/en/java/javase/25/docs/api/java.desktop/javax/swing/package-summary.html.

7. Oracle, *Java SE 25 API Specification: javax.tools package*, https://docs.oracle.com/en/java/javase/25/docs/api/java.compiler/javax/tools/package-summary.html.

8. Project source code, examples, README, and tests supplied in pseudocode-compiler-phase2; extended in this Phase 3 deliverable.

Appendix

Appendix A: Build and run instructions

Requirements: JDK 17 or newer. Run these commands from the extracted project folder:

1. javac -encoding UTF-8 -d out src/*.java

2. java -cp out CompilerVisualizer — start the live visualizer.

3. java -cp out Main examples/1_factorial.txt — compile and execute a pseudocode file.

4. java -cp out DiagnosticsCli examples/phase3_multiple_errors.txt — show independent semantic diagnostics.

5. java -cp out TestRunner tests — run the complete regression suite.

6. java -cp out PerformanceRunner 10 — repeat the performance measurement.

Appendix B: Recovery example source

```
LET first : int = missing;
LET second : boolean = 7;
LET third : int = "wrong";
LET safe : int = 4;
```

The recovery tool analyzes the top-level constructs, records an error, removes the failing construct, and retries with fresh semantic state. The valid final declaration remains available to subsequent analysis.

Appendix C: Key source files

- src/Lexer.java, src/Parser.java, and src/ASTNode.java: scanning, parsing, and syntax tree structures.
- src/SemanticAnalyzer.java and src/SymbolTable.java: type checks, declarations, function signatures, and scopes.
- src/CodeGenerator.java and src/Executor.java: Java generation, compilation, and execution.
- src/CompilerVisualizer.java: live desktop interface.
- src/DiagnosticsCli.java: recoverable semantic diagnostics.
- src/PerformanceRunner.java: timing harness.
- src/TestRunner.java and tests/: automated validation.